A JAX Ecosystem for Likelihood-Based Inference in HEP

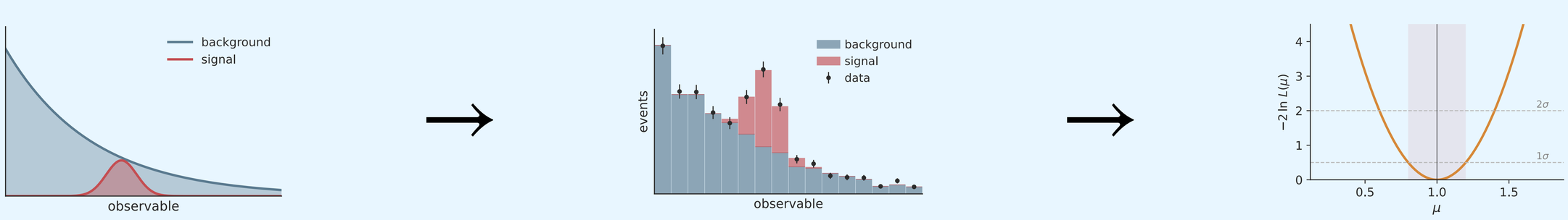
Mohamed Aly[1], Peter Fackeldey[1], Massimiliano Galli[1]
Princeton University[1]

Statistical Analyses in HEP

primitives
parameters . distributions. constraints

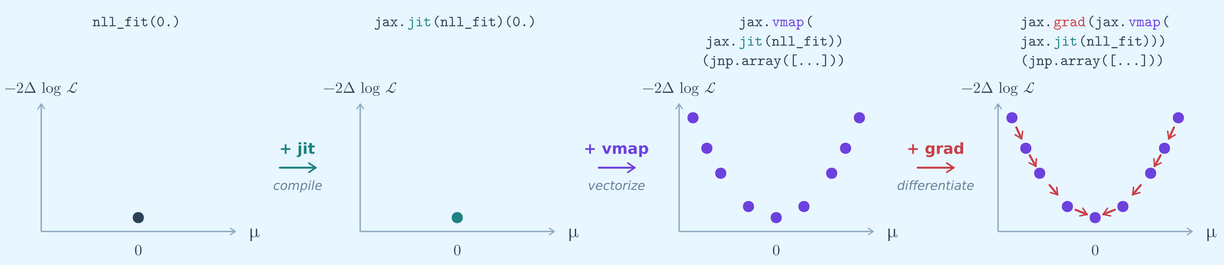
model
likelihoods . observations

inference
fits . intervals . limits



The case for

Transforms
compile . vectorise . differentiate . compose



Ecosystem
machine learning integration . debugging . inspection

Machine Learning	`flax.nnx` networks in JAX
Optimisation	`optimistix` differentiable solvers
Dataclasses	`equinox` modules as PyTrees
Checkpointing	`orbax` serialisation of states
Inspection	`mlflow` . `wandb` . `tensorboard` logging . monitoring

A JAX ecosystem for HEP statistics

Primitives

```python
import jax.numpy as jnp
import evermore as evm
hist = jnp.array([...])
syst = evm.NormalParameter(name="syst")
# lnN +/- 10% on the yield
modifier = syst.scale_log_asymmetric(
    up=jnp.array([1.1]),
    down=jnp.array([0.9]),
)
modified_hist = modifier(hist)
```

(parameters + constraints)

```python
import paramore as pm
lower, upper = 100.0, 180.0
sig = pm.Gaussian(mu=125.0, sigma=2.0,
                  lower=lower, upper=upper)
bkg = pm.Exponential(lambd=0.05,
                     lower=lower, upper=upper)
model = pm.SumPDF(
    pdfs=[sig, bkg],
    extended_vals=[500.0, 5000.0],
    lower=lower, upper=upper,
)
```

(distributions)

Quickstart

```python
import paramore as pm
import evermore as evm
# parameters (a PyTree)
params = {
    "sig_n": evm.Parameter(500.0),
    "bkg_n": evm.Parameter(5000.0),
    "mu":    evm.Parameter(125.0),
}
# extended NLL from params + model
def nll(p, data):
    return pm.create_extended_nll(p, model, data)
```

(unbinned)

```python
from jax.random import normal, key
import everwillow as ew
import everwillow.statelib as sl
# observed unbinned masses
data = normal(key(0), (10_000,)) * 2 + 125
# fit & parameter uncertainties
state  = sl.State.from_pytree(params)
obs    = {"data": data}
result = ew.fit(nll, state, obs)
unc    = ew.uncertainties(nll, result.params, obs)
```

(nll-agnostic)

Interoperability and likelihood combination

Differentiable Everything!
composable · introspectable · fast · ML-friendly

Likelihood Agnostic
all you need is

Acknowledgements: This work was supported by the National Science Foundation under Cooperative Agreement PHY-2323298 and grant OAC-2103945

Contact:
mohamed.aly@cern.ch